

Ejercicios básicos con John the Ripper (Kali Linux)

Ejercicio 1: Crear hash SHA1 y ficheros de prueba

```
# hash SHA1 de "hello" es: aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d
```

```
printf 'aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d\n' > hashes_sha1.txt
```

```
printf 'hello\n' > wordlist_hello.txt
```

```
# comprobar contenido
```

```
echo "HASHES:"
```

```
cat hashes_sha1.txt
```

```
echo
```

```
echo "WORDLIST:"
```

```
cat wordlist_hello.txt
```

Intento con John (usa potfile temporal para no tocar ~/.john/john.pot)

```
john --pot=~/temp_sha1.pot --format=raw-sha1 --wordlist=wordlist_hello.txt
```

```
hashes_sha1.txt
```

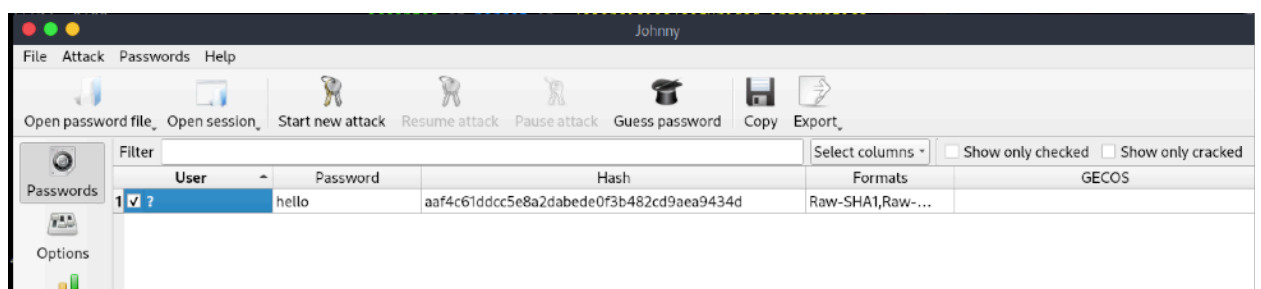
```
john --pot=~/temp_sha1.pot --show hashes_sha1.txt
```

```
jcsantos at parrot in ~/ejercicios/seguridad_informatica
└─○ sudo john --pot=~/temp_sha1.pot --format=raw-sha1 --wordlist=wordlist_hello.txt hashes_sha1.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA1 [SHA1 256/256 AVX2 8x])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 1 candidate left, minimum 8 needed for performance.
hello (???)
1g 0:00:00:00 DONE (2026-01-26 12:09) 100.0g/s 100.0p/s 100.0c/s 100.0C/s hello
Use the "--show --format=Raw-SHA1" options to display all of the cracked passwords reliably
Session completed.
└─○ jcsantos at parrot in ~/ejercicios/seguridad_informatica
```

Salida esperada en --show (ejemplo):

```
aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d:hello
```

```
1 password hash cracked, 0 left
```



The screenshot shows the JohnnY application window. The main table displays the results of the cracking process:

User	Password	Hash	Formats	GECOS
1 ✓ ?	hello	aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d	Raw-SHA1,Raw-...	

Additional details visible in the interface include the 'Filter' field, 'Select columns' dropdown, and checkboxes for 'Show only checked' and 'Show only cracked'.

Si prefieres hashcat (modo 100 = SHA1)

ejecutar cracking con hashcat (modo 100 = SHA1)

```
hashcat -m 100 hashes_sha1.txt wordlist_hello.txt --potfile-path=./  
hashcat_sha1.pot --show
```

Salida esperada:

aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d:hello

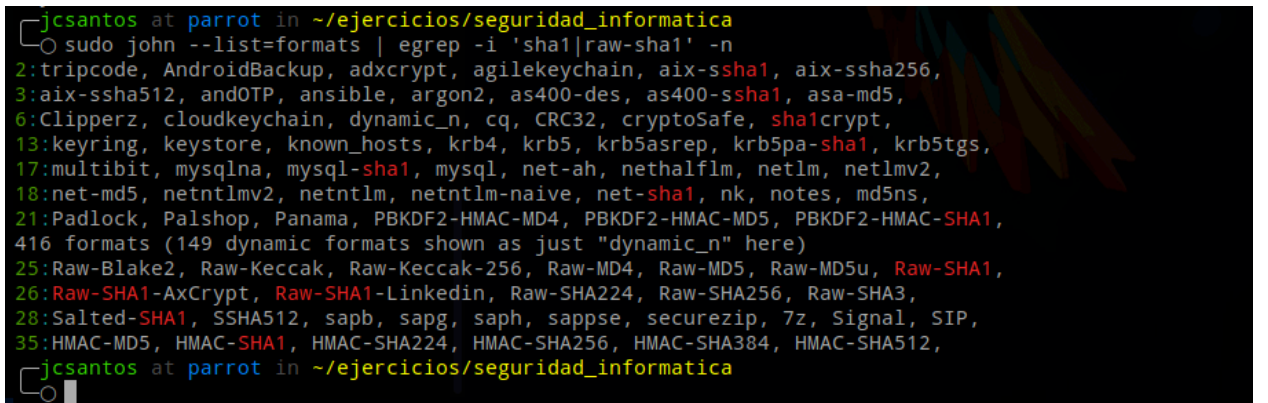
Diagnóstico rápido si John no muestra nada

ver contenido pot temporal (si usaste --pot=./temp_sha1.pot)

```
cat ./temp_sha1.pot 2>/dev/null || true
```

comprobar formato detectado por john (lista formatos)

```
john --list=formats | egrep -i 'sha1|raw-sha1' -n
```



```
jcsantos at parrot in ~/ejercicios/seguridad_informatica  
○ sudo john --list=formats | egrep -i 'sha1|raw-sha1' -n  
2:tripcode, AndroidBackup, adxencrypt, agilekeychain, aix-ssha1, aix-ssha256,  
3:aix-ssha512, andOTP, ansible, argon2, as400-des, as400-ssha1, asa-md5,  
6:Clipperz, cloudkeychain, dynamic_n, cq, CRC32, cryptoSafe, sha1crypt,  
13:keyring, keystore, known_hosts, krb4, krb5, krb5asrep, krb5pa-sha1, krb5tgs,  
17:multibit, mysqlna, mysql-sha1, mysql, net-ah, nethalflm, netlm, netlmv2,  
18:net-md5, netntlmv2, netntlm, netntlm-naive, net-sha1, nk, notes, md5ns,  
21:Padlock, Palshop, Panama, PBKDF2-HMAC-MD4, PBKDF2-HMAC-MD5, PBKDF2-HMAC-SHA1,  
416 formats (149 dynamic formats shown as just "dynamic_n" here)  
25:Raw-Blake2, Raw-Keccak, Raw-Keccak-256, Raw-MD4, Raw-MD5, Raw-MD5u, Raw-SHA1,  
26:Raw-SHA1-AxCrypt, Raw-SHA1-Linkedin, Raw-SHA224, Raw-SHA256, Raw-SHA3,  
28:Salted-SHA1, SHA512, sapb, sapg, saph, sappse, securezip, 7z, Signal, SIP,  
35:HMAC-MD5, HMAC-SHA1, HMAC-SHA224, HMAC-SHA256, HMAC-SHA384, HMAC-SHA512,  
jcsantos at parrot in ~/ejercicios/seguridad_informatica  
○
```

Si ves mensajes tipo "No password hashes left to crack" o "0 password hashes cracked, 1 left"

prueba usando un potfile nuevo (como en los ejemplos) para forzar prueba limpia.

asegúrate de que hashes_sha1.txt solo tenga el hash (una línea, sin espacios ni CRLF extras). Usa xxd hashes_sha1.txt para verificar

Ejercicio 2: Crear ficheros de prueba

hash SHA256 de "secret123"

```
printf 'fcf730b6d95236ecd3c9fc2d92d7b6b2bb061514961aec041d6c7a7192f592e4\n' >  
hashes_sha256.txt
```

wordlist con la contraseña en claro

```
printf 'secret123\n' > wordlist_secret.txt
```

comprobar contenido

```
echo "HASH:"
```

```
cat hashes_sha256.txt
```

```
echo
```

```
echo "WORDLIST:"
```

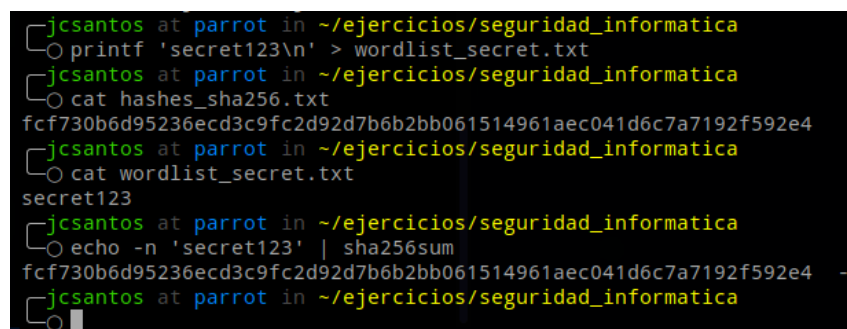
```
cat wordlist_secret.txt
```

2. Verificar localmente el hash (opcional)

```
echo -n 'secret123' | sha256sum
```

salida esperada:

```
fcf730b6d95236ecd3c9fc2d92d7b6b2bb061514961aec041d6c7a7192f592e4 -
```



```
jcsantos at parrot in ~/ejercicios/seguridad_informatica  
└─○ printf 'secret123\n' > wordlist_secret.txt  
jcsantos at parrot in ~/ejercicios/seguridad_informatica  
└─○ cat hashes_sha256.txt  
fcf730b6d95236ecd3c9fc2d92d7b6b2bb061514961aec041d6c7a7192f592e4  
jcsantos at parrot in ~/ejercicios/seguridad_informatica  
└─○ cat wordlist_secret.txt  
secret123  
jcsantos at parrot in ~/ejercicios/seguridad_informatica  
└─○ echo -n 'secret123' | sha256sum  
fcf730b6d95236ecd3c9fc2d92d7b6b2bb061514961aec041d6c7a7192f592e4 -  
jcsantos at parrot in ~/ejercicios/seguridad_informatica  
└─○
```

3. Ejecutar John (potfile temporal para no tocar ~/.john/john.pot)

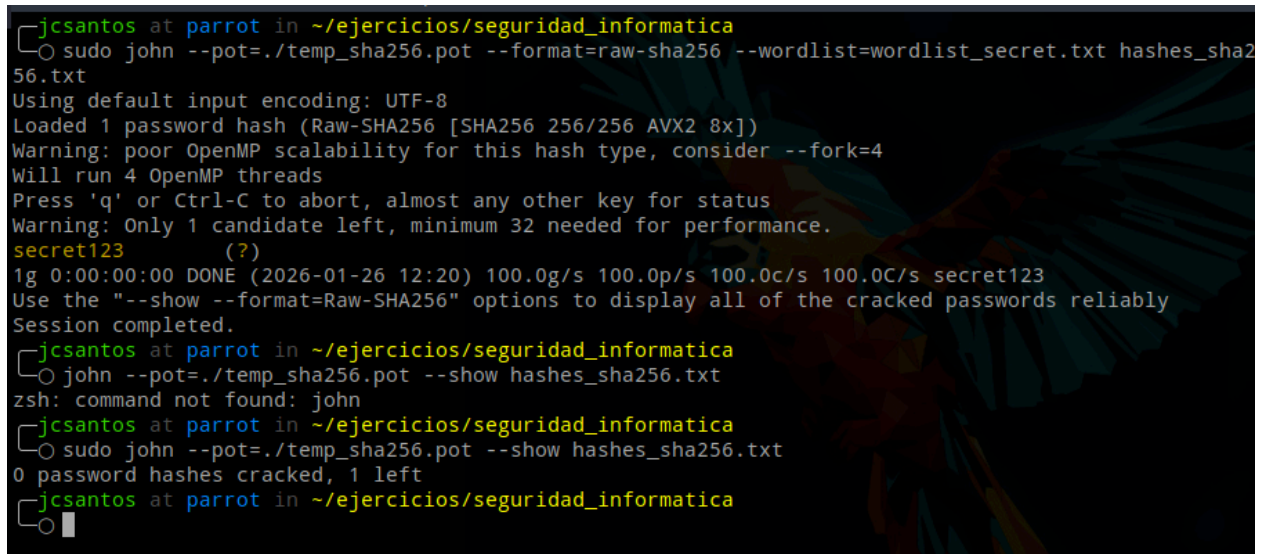
```
john --pot=./temp_sha256.pot --format=raw-sha256 --wordlist=wordlist_secret.txt  
hashes_sha256.txt
```

```
john --pot=./temp_sha256.pot --show hashes_sha256.txt
```

Salida esperada en --show:

fcf730b6d95236ecd3c9fc2d92d7b6b2bb061514961aec041d6c7a7192f592e4:secret123

1 password hash cracked, 0 left



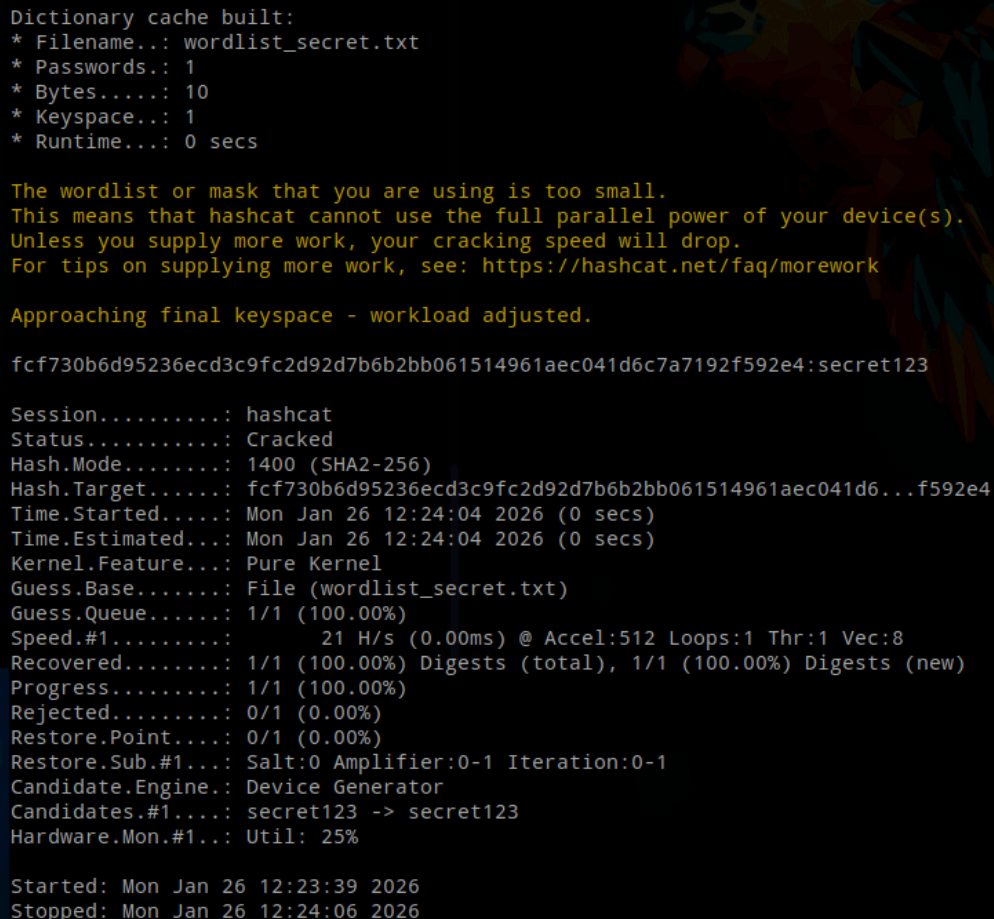
```
jcsantos at parrot in ~/ejercicios/seguridad_informatica  
○ sudo john --pot=./temp_sha256.pot --format=raw-sha256 --wordlist=wordlist_secret.txt hashes_sha256.txt  
Using default input encoding: UTF-8  
Loaded 1 password hash (Raw-SHA256 [SHA256 256/256 AVX2 8x])  
Warning: poor OpenMP scalability for this hash type, consider --fork=4  
Will run 4 OpenMP threads  
Press 'q' or Ctrl-C to abort, almost any other key for status  
Warning: Only 1 candidate left, minimum 32 needed for performance.  
secret123 (?)  
1g 0:00:00:00 DONE (2026-01-26 12:20) 100.0g/s 100.0p/s 100.0c/s 100.0C/s secret123  
Use the "--show --format=Raw-SHA256" options to display all of the cracked passwords reliably  
Session completed.  
jcsantos at parrot in ~/ejercicios/seguridad_informatica  
○ john --pot=./temp_sha256.pot --show hashes_sha256.txt  
zsh: command not found: john  
jcsantos at parrot in ~/ejercicios/seguridad_informatica  
○ sudo john --pot=./temp_sha256.pot --show hashes_sha256.txt  
0 password hashes cracked, 1 left  
jcsantos at parrot in ~/ejercicios/seguridad_informatica  
○
```

En el caso de que no salga nada

4. Alternativa con hashcat (modo 1400 = SHA-256)

correr hashcat (si lo tienes instalado)

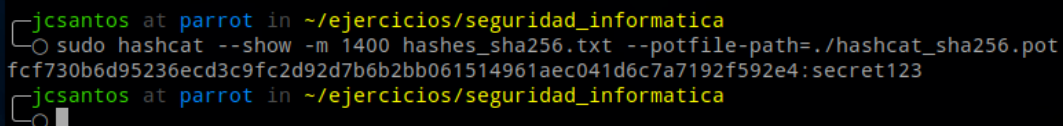
```
hashcat -m 1400 hashes_sha256.txt wordlist_secret.txt --potfile-path=./  
hashcat_sha256.pot
```

A terminal window with a dark background and a parrot illustration on the right. The output of the hashcat command is displayed in yellow and white text.

```
Dictionary cache built:  
* Filename.: wordlist_secret.txt  
* Passwords.: 1  
* Bytes.....: 10  
* Keyspace..: 1  
* Runtime...: 0 secs  
  
The wordlist or mask that you are using is too small.  
This means that hashcat cannot use the full parallel power of your device(s).  
Unless you supply more work, your cracking speed will drop.  
For tips on supplying more work, see: https://hashcat.net/faq/morework  
  
Approaching final keyspace - workload adjusted.  
  
fcf730b6d95236ecd3c9fc2d92d7b6b2bb061514961aec041d6c7a7192f592e4:secret123  
  
Session.....: hashcat  
Status.....: Cracked  
Hash.Mode.....: 1400 (SHA2-256)  
Hash.Target.....: fcf730b6d95236ecd3c9fc2d92d7b6b2bb061514961aec041d6...f592e4  
Time.Started.....: Mon Jan 26 12:24:04 2026 (0 secs)  
Time.Estimated...: Mon Jan 26 12:24:04 2026 (0 secs)  
Kernel.Feature...: Pure Kernel  
Guess.Base.....: File (wordlist_secret.txt)  
Guess.Queue.....: 1/1 (100.00%)  
Speed.#1.....: 21 H/s (0.00ms) @ Accel:512 Loops:1 Thr:1 Vec:8  
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)  
Progress.....: 1/1 (100.00%)  
Rejected.....: 0/1 (0.00%)  
Restore.Point...: 0/1 (0.00%)  
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1  
Candidate.Engine.: Device Generator  
Candidates.#1...: secret123 -> secret123  
Hardware.Mon.#1..: Util: 25%  
  
Started: Mon Jan 26 12:23:39 2026  
Stopped: Mon Jan 26 12:24:06 2026
```

mostrar resultados

```
hashcat --show -m 1400 hashes_sha256.txt --potfile-path=./hashcat_sha256.pot
```

A terminal window showing the command and its output.

```
jcsantos at parrot in ~/ejercicios/seguridad_informatica  
○ sudo hashcat --show -m 1400 hashes_sha256.txt --potfile-path=./hashcat_sha256.pot  
fcf730b6d95236ecd3c9fc2d92d7b6b2bb061514961aec041d6c7a7192f592e4:secret123  
jcsantos at parrot in ~/ejercicios/seguridad_informatica  
○
```

5. Diagnóstico — si John no muestra nada

comprobar si hay caracteres invisibles / CRLF

```
xxd hashes_sha256.txt
```

```
cat -v hashes_sha256.txt
```

```
# normalizar por si hay CRLF
```

```
dos2unix hashes_sha256.txt 2>/dev/null || sed -i 's/\r$//' hashes_sha256.txt
```

```
# listar formatos disponibles en John
```

```
john --list=formats | egrep -i 'sha256|raw-sha256' -n
```

6. Si quieres un único comando para todo (prueba rápida con John)

```
printf 'fcf730b6d95236ecd3c9fc2d92d7b6b2bb061514961aec041d6c7a7192f592e4\n' > h.txt
```

```
printf 'secret123\n' > w.txt
```

```
john --pot=./tmp.pot --format=raw-sha256 --wordlist=w.txt h.txt && john --pot=./tmp.pot --show h.txt
```

Si lo ejecutas y te sale algo raro, pega aquí la salida completa de:

```
john --pot=./temp_sha256.pot --format=raw-sha256 --wordlist=wordlist_secret.txt hashes_sha256.txt
```

```
john --pot=./temp_sha256.pot --show hashes_sha256.txt
```

```
xxd hashes_sha256.txt | sed -n '1,5p'
```

Ejercicio 3: Ejemplo SHA-256

Perfecto — otro ejemplo SHA-256, paso a paso. Usaré la contraseña MiClave!2025. Su SHA-256 es:

c0f2bd9f0e58d2395799ee0b4802c8a1e28c8a4665af33e785c5cd4d255d75be

Copia/pega estos comandos en tu terminal (Kali/Parrot):

1. Crear ficheros de prueba

```
printf 'c0f2bd9f0e58d2395799ee0b4802c8a1e28c8a4665af33e785c5cd4d255d75be\n' > hashes_sha256_b.txt
```

```
printf 'MiClave!2025\n' > wordlist_miclave.txt
```

2. Verificar localmente el hash (opcional)

```
echo -n 'MiClave!2025' | sha256sum
```

salida esperada:

c0f2bd9f0e58d2395799ee0b4802c8a1e28c8a4665af33e785c5cd4d255d75be -

A screenshot of a terminal window with a dark background and a parrot watermark. The window title is 'jcsantos@parrot:~/ejercicios/seguridad_informatica'. The terminal shows a series of commands and their outputs. The first command is 'printf 'c0f2bd9f0e58d2395799ee0b4802c8a1e28c8a4665af33e785c5cd4d255d75be\n' > hashes_sha256_b.txt'. The second is 'printf 'MiClave!2025\n' > wordlist_miclave.txt'. The third is 'echo -n 'MiClave!2025' | sha256sum', which outputs 'c0f2bd9f0e58d2395799ee0b4802c8a1e28c8a4665af33e785c5cd4d255d75be -'. The fourth is 'cat hashes_sha256_b.txt', which outputs the same hash. The terminal prompt is 'jcsantos at parrot in ~/ejercicios/seguridad_informatica'.

3. Probar con John (potfile temporal para no tocar ~/.john/john.pot)

```
john --pot=./temp_sha256_b.pot --format=raw-sha256 --wordlist=wordlist_miclave.txt hashes_sha256_b.txt
```

```
john --pot=./temp_sha256_b.pot --show hashes_sha256_b.txt
```

Salida esperada en --show:

```
c0f2bd9f0e58d2395799ee0b4802c8a1e28c8a4665af33e785c5cd4d255d75be:MiClave!  
2025
```

1 password hash cracked, 0 left

```
jcsantos at parrot in ~/ejercicios/seguridad_informatica  
○ sudo john --pot=./temp_sha256_b.pot --format=raw-sha256 --wordlist=wordlist_miclave.txt hashes_s  
ha256_b.txt  
Using default input encoding: UTF-8  
Loaded 1 password hash (Raw-SHA256 [SHA256 256/256 AVX2 8x])  
Warning: poor OpenMP scalability for this hash type, consider --fork=4  
Will run 4 OpenMP threads  
Press 'q' or Ctrl-C to abort, almost any other key for status  
Warning: Only 1 candidate left, minimum 32 needed for performance.  
MiClave!2025 (?)  
1g 0:00:00:00 DONE (2026-01-26 12:36) 100.0g/s 100.0p/s 100.0c/s 100.0C/s MiClave!2025  
Use the "--show --format=Raw-SHA256" options to display all of the cracked passwords reliably  
Session completed.  
jcsantos at parrot in ~/ejercicios/seguridad_informatica  
○
```

4. Alternativa con hashcat (modo 1400 = SHA-256)

```
hashcat -m 1400 hashes_sha256_b.txt wordlist_miclave.txt --potfile-path=./  
hashcat_sha256_b.pot
```

```
hashcat --show -m 1400 hashes_sha256_b.txt --potfile-path=./hashcat_sha256_b.pot
```

```
The wordlist or mask that you are using is too small.  
This means that hashcat cannot use the full parallel power of your device(s).  
Unless you supply more work, your cracking speed will drop.  
For tips on supplying more work, see: https://hashcat.net/faq/morework  
  
Approaching final keypace - workload adjusted.  
  
c0f2bd9f0e58d2395799ee0b4802c8a1e28c8a4665af33e785c5cd4d255d75be:MiClave!2025  
  
Session.....: hashcat  
Status.....: Cracked  
Hash.Mode.....: 1400 (SHA2-256)  
Hash.Target.....: c0f2bd9f0e58d2395799ee0b4802c8a1e28c8a4665af33e785c...5d75be  
Time.Started.....: Mon Jan 26 12:38:13 2026 (0 secs)  
Time.Estimated...: Mon Jan 26 12:38:13 2026 (0 secs)  
Kernel.Feature...: Pure Kernel  
Guess.Base.....: File (wordlist_miclave.txt)  
Guess.Queue.....: 1/1 (100.00%)  
Speed.#1.....: 26 H/s (0.00ms) @ Accel:512 Loops:1 Thr:1 Vec:8  
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)  
Progress.....: 1/1 (100.00%)  
Rejected.....: 0/1 (0.00%)  
Restore.Point....: 0/1 (0.00%)  
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1  
Candidate.Engine.: Device Generator  
Candidates.#1....: MiClave!2025 -> MiClave!2025  
Hardware.Mon.#1...: Util: 25%  
  
Started: Mon Jan 26 12:38:08 2026  
Stopped: Mon Jan 26 12:38:15 2026  
jcsantos at parrot in ~/ejercicios/seguridad_informatica  
○
```


5. Diagnóstico rápido (si John/Hashcat no muestran nada)

comprobar CRLF / caracteres invisibles

```
xxd hashes_sha256_b.txt
```

```
cat -v hashes_sha256_b.txt
```

normalizar si hay CRLF

```
dos2unix hashes_sha256_b.txt 2>/dev/null || sed -i 's/\r$//' hashes_sha256_b.txt
```

listar formatos disponibles en John

```
john --list=formats | egrep -i 'sha256|raw-sha256' -n
```

6. Comando todo-en-uno (prueba rápida)

```
printf 'c0f2bd9f0e58d2395799ee0b4802c8a1e28c8a4665af33e785c5cd4d255d75be\n'  
> h.txt
```

```
printf 'MiClave!2025\n' > w.txt
```

```
john --pot=./tmp_b.pot --format=raw-sha256 --wordlist=w.txt h.txt && john --pot=./  
tmp_b.pot --show h.txt
```

Si lo ejecutas y obtienes salida rara, pégame aquí la salida completa de:

```
john --pot=./temp_sha256_b.pot --format=raw-sha256 --wordlist=wordlist_miclave.txt  
hashes_sha256_b.txt
```

```
xxd hashes_sha256_b.txt | sed -n '1,5p'
```